A Project Report

on

"InternHub – Remote Internship Progress Dashboard"

AKS UNIVERSITY, SATNA

B.Tech, CSE (AI & DS), 6th Semester

Submitted by

Amratansh Pandey (B2355R10195116)

Program : M.P. ONLINE INTERNSHIP

Department of Computer Science & Engineering

AKS University, Satna (M.P.)

CERTIFICATE

This is to certify that the project report entitled "InternHub – Remote Internship Progress Dashboard" submitted by "Amratansh Pandey" in partial fulfillment of the requirements for the award of the degree of "Bachelor of Technology in Computer Science and Engineering" at AKS University, Satna, is a bona fide record of project work carried out under the guidance and supervision of the undersigned.

The work presented in this report is original and has not been submitted, either in part or in full, for the award of any other degree or diploma to any other institution or university.

All assistance and sources of information used during the development of the project have been duly acknowledged in the report.

Place: AKSU, Satna

Date: ___ / ___ / _____

External Signature: _____

Guide Signature: _____

Head of Department: _____

Dean: _____

SELF DECLARATION

I hereby declare that the work presented in this project report entitled "InternHub – Remote Internship Progress Dashboard" is an authentic record of my own work carried out in partial fulfillment of the requirements for the award of the degree of "B.Tech" in the Department of Computer Science and Engineering, AKS University, Satna (M.P.).

I further declare that this work has not been submitted earlier for the award of any degree or diploma to any other institute or university.

Signature of Candidate: _____

Name: Amratansh Pandey

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Mr. Dhruv Pandey, and the Department of Computer Science and Engineering, AKS University, Satna, for their guidance, support, and encouragement throughout the development of this project.

I am especially thankful to my guide for providing valuable suggestions during the planning, design, implementation, and evaluation stages of the project. Their academic and technical support helped shape the direction of this work and significantly improved the quality of the final system.

I would also like to thank my parents and friends for their continuous moral support, motivation, and encouragement. Their confidence in me helped me stay focused and complete the project successfully.

ABSTRACT

"InternHub – Remote Internship Progress Dashboard" is a full-stack web application designed to help organizations, mentors, and interns manage remote internship workflows in a centralized, structured, and efficient manner. Traditional internship monitoring often relies on scattered email threads, spreadsheets, and disconnected messaging applications, making it difficult for mentors to track task progress, review reports, and provide timely feedback.

This project addresses that problem by integrating role-based user authentication, task management with a Kanban board, markdown-based weekly report submission, and a mentor feedback module into one unified platform. The application is built using React with react-kanban for the frontend and FastAPI for the backend, while SQLAlchemy handles database interaction with SQLite as the persistence layer. JWT-based authentication provides secure role-based access for Admin, Mentor, and Intern user roles.

The core strength of the project is its Kanban-based task management board, which allows mentors to assign tasks to interns and track their status visually, and its structured report submission system, which enables interns to submit weekly progress in markdown format. Mentors can then review submissions and provide structured feedback, creating a complete internship management cycle within one platform.

The system is modular, scalable, and suitable for internship management use. It demonstrates the practical integration of modern web technologies to solve a real-world remote work coordination problem through structured planning, role-based access, and collaborative feedback mechanisms.

Keywords: Remote Internship Dashboard, Kanban Board, FastAPI, React, SQLAlchemy, SQLite, JWT Authentication, Role-Based Access Control, Report Submission, Mentor Feedback.

TABLE OF CONTENTS

1. Introduction 6
2. Objective of The Project 8
3. Scope of The Project 10
4. Problem Definition 13
5. Benefits of The Project 15
6. Project Modules 17
7. Technical Overview 21
8. System Configuration 27
9. Project Explanation 31
10. Output Screens 33
11. Result and Analysis 35
12. Software Testing 38
13. Conclusion 41
14. References 43

Introduction

The rapid expansion of remote work, digital collaboration tools, and online internship programs has created a strong demand for systems that can help organizations manage intern progress efficiently. In many companies and academic institutions, internship programs involve multiple participants—admins, mentors, and interns—who must coordinate across task assignments, weekly progress updates, and structured feedback cycles. However, most organizations still depend on scattered tools such as email threads, shared spreadsheets, or messaging applications to run these workflows. These methods are often inconsistent, difficult to track, and not connected to actual progress data.

A Remote Internship Progress Dashboard provides a structured digital environment where all internship-related activities can be managed in one place. In its modern form, such a platform is not limited to simple task lists. It can support kanban-based visual task management, structured report submission, role-based access control, and mentor feedback mechanisms, transforming it from a basic tracking tool into a comprehensive internship management system.

The present project, InternHub – Remote Internship Progress Dashboard, has been developed to address this need. It allows users to register and log in with role-specific permissions, manage and assign tasks through a Kanban board interface, submit weekly markdown-formatted progress reports, and receive structured feedback from mentors. The system also includes a centralized dashboard that gives each user a relevant overview of their internship activities based on their role.

This project combines the strengths of modern frontend and backend technologies. The frontend is developed using React with the react-kanban library, providing an interactive and responsive user interface. The backend is built using FastAPI, a lightweight and high-performance Python web framework designed for REST API development. SQLAlchemy handles database interaction, and SQLite serves as the auto-created persistent data store. JWT-based authentication enforces secure role-based access for Admin, Mentor, and Intern user types.

One of the most useful features of this project is the Kanban task board, which gives a visual representation of task status across columns. Combined with the markdown report submission system and the mentor feedback module, the platform covers the complete internship management cycle within one cohesive application.

The project is designed for simplicity, modularity, and practical value. It can be used by technology companies, academic institutions, and training programs that need a centralized system for managing remote internships. The modular structure also makes it suitable for future extension with features such as email notifications, real-time updates, advanced analytics, calendar integration, and file upload support.

In summary, InternHub is a practical attempt to combine authentication, task management, report submission, and feedback into one coherent internship productivity platform. It demonstrates how full-stack development can be applied to solve a meaningful real-world coordination problem in remote academic and professional environments.

Objective of The Project

The primary objective of "InternHub – Remote Internship Progress Dashboard" is to design and develop a modern, web-based internship management platform that helps organizations, mentors, and interns coordinate their activities in a systematic, secure, and efficient manner. The system aims to reduce the difficulties faced during remote internship programs, where participants must manage task assignments, progress reporting, and feedback across multiple tools without a unified environment.

Another major objective of the project is to improve task visibility and accountability. In many internship programs, task assignments are communicated informally through emails or chat messages, which makes it difficult to track status, identify bottlenecks, and evaluate intern productivity. This project addresses that issue by providing a Kanban board interface where tasks can be created, assigned, and moved through defined stages visually.

The project also aims to introduce structured reporting and feedback workflows. Through the weekly report submission module, interns can document progress in a markdown-formatted report that mentors can review and respond to with structured feedback. This creates a clear, auditable record of internship activity and encourages consistent communication between participants.

From a technical perspective, the objective is to implement a complete full-stack software solution using modern technologies such as React, FastAPI, SQLAlchemy, and SQLite. The project demonstrates how frontend design, backend API architecture, authentication, and database design can be integrated into one real-world application.

Specific Objectives

- To design a secure user authentication system with signup, login, and role-based access control for Admin, Mentor, and Intern roles.
- To create a centralized dashboard where each user can view activity relevant to their role.
- To provide a Kanban board interface for visual task management and status tracking.
- To allow mentors to assign tasks to interns and track task completion efficiently.
- To enable interns to submit structured weekly progress reports using markdown format.
- To provide mentors with the ability to add, view, and track feedback on submitted reports.

- To build a modular backend using FastAPI that cleanly separates authentication, task, report, and feedback logic.
- To implement a scalable project structure that supports future improvements and additional modules.
- To provide a simple, responsive, and user-friendly interface accessible to users with basic technical knowledge.
- To demonstrate practical application of full-stack development concepts in an internship management domain.

Scope of The Project

The scope of "InternHub – Remote Internship Progress Dashboard" includes the design and implementation of a complete internship management platform that supports task coordination, report submission, mentor feedback, and role-based user access. The project is intended to solve the coordination challenges faced during remote internship programs, where admins, mentors, and interns need a common platform to manage their activities without depending on scattered communication tools.

This project primarily focuses on the intern-mentor workflow. It begins from user registration and secure login with role-based permissions, extends to task management through a Kanban board, and reaches up to report submission and mentor feedback. In this way, the scope is not limited to simple data entry or record keeping. Instead, it includes structured workflows for each user role, making the application genuinely useful for managing remote internship programs.

Current Functional Scope

- User registration, login, logout, and role-based access for Admin, Mentor, and Intern.
- A centralized dashboard providing role-specific overview of activities.
- Kanban board for task creation, assignment, status updates, and deletion.
- Task assignment from mentors to specific interns.
- Weekly report submission by interns in markdown format.
- Mentor ability to view, review, and respond to submitted reports.
- Feedback tracking with history per report.
- A modular backend API with organized routers for authentication, tasks, reports, and feedback.
- SQLite-based data persistence with auto-creation on application startup.

Technical Scope

The scope also includes the technical development of a full-stack application using modern web technologies. On the frontend side, the project covers component-based UI design using React, Kanban board rendering using react-kanban, and centralized API communication through a dedicated service layer. On the backend side, it includes REST API development using FastAPI, schema validation using Pydantic, database modeling using SQLAlchemy, and JWT-based authentication handling.

Intended Users

- Companies running structured remote internship programs.
- Academic institutions managing student internship workflows.
- Training organizations tracking learner task progress and weekly updates.
- Mentors who need a centralized view of intern activities and report submissions.

Future Extension Potential

- Email or push notifications for task assignments and
- In the current remote work environment, internship programs often involve participants spread across different locations with varying schedules and report submissions.
- Real-time updates using WebSockets.
- Advanced analytics and reporting dashboards.
- File upload and attachment support.
- Calendar synchronization for deadlines and milestones.
- Dark mode support and mobile-responsive design improvements.

Current Limitations

The present version does not provide real-time messaging, video calling, file attachment support, or institution-wide management controls. It is not intended to replace a full enterprise HR or project management platform at this stage. Instead, it is designed as a focused internship tracking tool with clear role-based workflows and structured reporting. These limitations identify clear directions for future enhancement.

communication preferences. Admins need to manage user roles, mentors need to assign and review tasks, and interns need to report progress and receive feedback. Without a unified platform, these activities become fragmented across email inboxes, chat applications, spreadsheets, and phone calls.

One of the most common problems is the fragmentation of internship data. Task assignments, progress updates, report submissions, and feedback responses are maintained in different places. Since these tools are not integrated, mentors may miss submitted reports, interns may not receive timely feedback, and admins may not have a clear picture of overall internship activity.

Another major problem is the lack of structured task tracking. In many internship setups, tasks are assigned informally and their status is tracked manually. There is no visual representation of what tasks are in progress, what is blocked, and what has been completed. This leads to poor accountability, duplicated effort, and missed deadlines.

A deeper issue is the absence of structured reporting workflows. Interns are often expected to provide weekly updates, but without a formal system, these updates may be inconsistent in format, delayed in submission, or difficult to search and review later. Mentors cannot efficiently provide feedback on reports that arrive through informal channels.

There is also a clear problem in terms of role-based access control. Many general-purpose tools do not enforce strict role separation. When admins, mentors, and interns all share the same interface without role-specific views, the experience becomes confusing and important information may be exposed to the wrong participants.

Summary of Identified Problems

- No centralized platform to manage tasks, reports, and feedback for remote internship programs.
- Fragmentation of internship data across email, spreadsheets, and chat tools.
- No visual task tracking mechanism to identify progress and blockers.
- Absence of a formal, structured weekly report submission system.
- Lack of organized mentor feedback mechanism linked to specific reports.
- Poor role separation causing confusion between admin, mentor, and intern responsibilities.
- Difficulty in maintaining an auditable record of internship activities.
- Higher chances of missed feedback, delayed responses, and inconsistent progress tracking.

These problems directly affect the quality and effectiveness of remote internship programs. To address this challenge, there is a need for a system that can enforce role-based access, organize tasks visually, support structured report submission, and provide a feedback mechanism linked directly to submitted reports. InternHub – Remote Internship Progress Dashboard is proposed as a solution to this problem.

Benefits of The Project

"InternHub – Remote Internship Progress Dashboard" provides several important benefits to organizations, mentors, and interns by combining task management, structured reporting, and role-based access into one integrated platform. The project is designed not only to store information, but also to enable better coordination, clearer accountability, and more structured workflows in remote internship programs.

Key Benefits

- Centralized management: All tasks, reports, and feedback are accessible in one platform, eliminating the need for scattered tools and informal communication channels.
- Visual task tracking: The Kanban board provides an instant overview of task status, helping mentors and interns identify progress and blockers at a glance.
- Structured reporting: Interns can submit consistently formatted weekly reports, creating an auditable record of internship activity and making it easier for mentors to review progress.
- Organized mentor feedback: Feedback is linked directly to specific reports, creating a clear conversation thread for each submission and improving the quality of intern guidance.
- Role-based access control: Admins, mentors, and interns see only what is relevant to their role, reducing confusion and improving the security of sensitive internship data.
- Improved accountability: Clear task assignments with owner information and status tracking make it easy to identify which intern is responsible for which deliverable.
- Reduced coordination overhead: Mentors no longer need to chase updates through email or chat, as all progress is documented in one location.
- Easy onboarding: New interns can register and access their tasks and reports immediately, with no manual configuration required for the database (SQLite is auto-created).
- Scalable architecture: The modular backend allows future features such as notifications, calendar integration, and analytics to be added without redesigning the system.
- Practical full-stack example: The project serves as a strong academic demonstration of React, FastAPI, SQLAlchemy, and JWT authentication working together in a real-world scenario.

From a broader perspective, the project also has technical and developmental benefits. It demonstrates the practical use of full-stack development concepts such as frontend-backend integration, REST API communication, role-based authentication, database design, and Kanban

UI rendering. Therefore, the system is beneficial not only to end users, but also as a strong academic software engineering case study in remote workflow management.

Project Modules

The proposed system is divided into several functional modules so that each part of the application handles a specific responsibility in an organized and maintainable way. This modular design improves readability, testing, and future scalability of the project. Each module performs an independent role, but all of them work together to provide a complete internship management experience.

1. Authentication Module

The Authentication Module is responsible for handling user identity and securing access to the system. It manages the registration and login process, verifies user credentials, creates secure JWT access tokens, and enforces role-based access control for Admin, Mentor, and Intern roles.

Passwords are stored in hashed form using bcrypt so that sensitive credentials remain protected. Once the user logs in successfully, a JWT token is issued and stored on the frontend (localStorage). This token is used to authorize all subsequent requests, ensuring that each user can only access data and functionality appropriate to their role.

2. User & Role Management Module

The User and Role Management Module provides the structure for handling different types of participants in the internship program. The system supports three distinct roles: Admin, who manages overall system access; Mentor, who assigns tasks and reviews reports; and Intern, who completes tasks and submits weekly reports.

Role-based routing on both the frontend and backend ensures that each participant sees only the information and actions relevant to their role. This module forms the access control foundation for all other modules in the system.

3. Task Management Module (Kanban Board)

The Task Management Module provides a visual Kanban board interface for managing internship tasks. Mentors can create tasks, assign them to specific interns, and track their progress through status columns. Interns can update task status and mark tasks as complete.

The Kanban board is implemented using the react-kanban library on the frontend, which enables drag-and-drop interactions and column-based status tracking. The backend handles task creation, retrieval, update, and deletion through a dedicated API router.

4. Report Submission Module

The Report Submission Module enables interns to submit structured weekly progress reports in markdown format. Each report is timestamped and associated with the submitting intern. Mentors can view all submitted reports and filter them by intern or submission date.

This module creates a consistent, searchable record of intern progress that replaces informal email updates. It also provides the entry point for the feedback cycle managed by the Feedback Module.

5. Mentor Feedback Module

The Mentor Feedback Module allows mentors to add structured feedback to submitted reports. Each feedback entry is linked to a specific report and includes the mentor's response. Interns can view the feedback they have received on each of their reports.

This module improves the quality of intern guidance by providing a formal feedback mechanism rather than informal chat messages. Feedback history is maintained per report, creating a clear progression of mentor-intern communication.

6. Dashboard Module

The Dashboard Module serves as the main overview screen of the application. Immediately after login, users see a role-specific summary of their internship activities. Mentors see task assignments and report submissions. Interns see their assigned tasks and submitted reports. Admins have an overview of all users and system activity.

This module improves usability by reducing navigation effort and helping users immediately understand their current priorities and pending actions.

7. API Service Module

The API Service Module works as the communication bridge between the frontend and backend. Implemented in the frontend as api.js, it centralizes all HTTP request logic, attaches JWT tokens to authorized requests, and manages error handling for API responses. This keeps the component-level code clean and reusable.

8. Database and Data Management Module

The Database and Data Management Module handles all data persistence using SQLite as the database and SQLAlchemy as the ORM layer. The database is auto-created on first application startup, requiring no manual database configuration. Core data models include User, Task, Report, and Feedback, which are connected through defined relationships.

9. Overall Role of Modules

- Authentication Module: Secures access and manages JWT tokens.
- User & Role Module: Enforces role-based access for Admin, Mentor, and Intern.
- Task Management Module: Provides visual Kanban-based task tracking.
- Report Submission Module: Enables structured weekly progress documentation.
- Feedback Module: Connects mentor responses to specific intern reports.
- Dashboard Module: Provides a role-specific overview of internship activity.
- API Service Module: Centralizes frontend-backend communication.
- Database Module: Manages persistent storage and relational data integrity.

Technical Overview

"InternHub – Remote Internship Progress Dashboard" is developed using a modern client-server full-stack architecture. The frontend handles user interaction and presentation, the backend manages application logic and request processing, and the database stores all persistent internship data. This separation of concerns makes the system organized, maintainable, and scalable.

1. Architectural Approach

The application follows a Modular Monolith architecture with a Layered Architecture design pattern. The three primary layers are: the Presentation Layer for the React-based user interface, the Application Layer for FastAPI request handling and business logic, and the Data Layer for SQLAlchemy-based database interaction.

The application flow is: Frontend (React UI) → API Calls (HTTP/JSON) → FastAPI Backend → Service Layer (Business Logic) → Database (SQLAlchemy ORM) → Response → Frontend Rendering.

2. Frontend Technology Overview

The frontend is built using React, a component-based JavaScript library for building dynamic user interfaces. The project structure separates pages (Login, Register, Dashboard, TaskBoard, Reports) from reusable components (Navbar, DashboardNavbar) and a centralized API service layer (api.js).

- React: Component-based UI library for building dynamic interfaces.
- react-kanban: Library for rendering the drag-and-drop Kanban task board.
- JavaScript (ES6+): Core scripting language for frontend logic.
- CSS: Custom styling for page layout and component presentation.
- api.js: Centralized HTTP request handler that manages all API communication.

3. Backend Technology Overview

The backend is developed using FastAPI, a modern Python web framework designed for building high-performance REST APIs. FastAPI is appropriate for this project because of its clean route definitions, automatic data validation through Pydantic, and strong typing support.

- FastAPI: High-performance Python web framework for REST API development.
- SQLAlchemy: ORM layer for database model definition and interaction.
- Pydantic: Request and response schema validation.
- python-jose: JWT token creation and verification.
- bcrypt: Secure password hashing for user authentication.
- Uvicorn: ASGI server for running the FastAPI application.
- SQLite: Lightweight relational database, auto-created on startup.

4. API Design and Communication

The system follows a RESTful API communication model. The frontend sends HTTP requests to backend endpoints, and the backend responds with JSON-formatted data. The API is organized into dedicated routers for authentication, tasks, reports, and feedback. The api.js service on the frontend attaches JWT tokens automatically to all authorized requests.

5. Database Overview

The project uses SQLite as the relational database. SQLite is auto-created at application startup through SQLAlchemy model definitions, eliminating the need for manual database setup. The primary tables are users, tasks, reports, and feedback, connected through foreign key relationships.

- One user can have many tasks (assigned to them).
- One user (intern) can submit many reports.
- One report can have many feedback entries from mentors.

6. Security Overview

- Password hashing using bcrypt before storage.
- JWT-based authentication for secure, stateless sessions.
- Protected API routes that require a valid token.
- Role-based access control enforced on both frontend routing and backend endpoints.
- Pydantic validation prevents malformed data from reaching the database layer.

7. Working Principle

The overall technical workflow of the project can be summarized as follows:

- A user registers through the registration page with their role selection.
- Login validates credentials and returns a JWT token stored in localStorage.
- The frontend attaches this token to all subsequent API requests via api.js.
- Mentors create tasks on the TaskBoard and assign them to interns.
- Interns view assigned tasks, update status, and submit weekly reports.
- Mentors review submitted reports and add structured feedback.
- The Dashboard presents a role-specific summary of all relevant activity.

System Configuration

The system configuration of "InternHub – Remote Internship Progress Dashboard" includes the hardware, software, development tools, and supporting libraries required to develop, run, and test the application. Since the project is a full-stack web application, its configuration covers both frontend and backend environments along with database support.

1. Hardware Requirements

- Processor: Intel Core i3 / AMD equivalent or above.
- RAM: Minimum 4 GB; recommended 8 GB or more.
- Storage: At least 1 GB free disk space for project files and dependencies.
- Display: Standard laptop or desktop display.
- Internet Connection: Required for initial dependency installation.

2. Software Requirements

- Operating System: Windows 10/11, Linux, or macOS.
- Python: Version 3.11 or above (for FastAPI backend).
- Node.js: Version 18 LTS or above (for React frontend).
- SQLite: Auto-created; no manual installation required.
- Web Browser: Google Chrome, Microsoft Edge, or Mozilla Firefox.

3. Development Environment

- Visual Studio Code as the primary code editor.
- Git for version control and repository management.
- Postman or browser tools for API endpoint testing.
- Terminal / Command Prompt / PowerShell for running servers.

4. Frontend Configuration

- React for component-based UI development.
- react-kanban for Kanban board rendering.
- JavaScript (ES6+) for frontend application logic.
- CSS for layout and component styling.
- npm for package management and development server.

5. Backend Configuration

- FastAPI for REST API development and routing.
- SQLAlchemy for ORM-based database management.
- Pydantic for data validation and schema enforcement.
- python-jose for JWT token generation and verification.
- bcrypt for password hashing.
- Uvicorn as the ASGI server for running the FastAPI application.

6. Database Configuration

The application uses SQLite as the database. The database file (internhub.db) is auto-created by SQLAlchemy on first startup using the defined model classes. No separate database server installation is required. Tables for users, tasks, reports, and feedback are automatically created through model definitions.

7. Quick Start Execution

- On Windows: Double-click start.bat for a one-click start.
- On Mac/Linux: Run bash start.sh in the project root.
- Manual: Run backend using uvicorn and frontend using npm run dev in separate terminals.
- Access the application at http://localhost:5173 in a web browser.

Project Explanation

InternHub is a student and mentor-oriented web application that combines task management, structured reporting, and role-based access control into one internship coordination platform.

1. User Authentication and Role Access

Users register through the Register page by providing their email, username, password, and role. After successful registration, they log in using email and password. The backend validates credentials, generates a JWT token, and returns it to the frontend. Protected routes such as Dashboard, TaskBoard, and Reports are accessible only after successful authentication. Role-based routing ensures Admins, Mentors, and Interns see content appropriate to their role.

2. Dashboard

After login, the user lands on the Dashboard, which provides a role-specific overview of internship activity. Mentors can see task assignments and submitted reports. Interns see their assigned tasks and submitted reports. The navigation bar (DashboardNavbar) allows quick access to the TaskBoard and Reports pages, and includes a logout option.

3. Task Management (Kanban Board)

The TaskBoard page renders a Kanban board using the react-kanban library. Mentors can create new tasks, assign them to specific interns, and track their status across visual columns. Interns can view their assigned tasks and update task status to reflect progress. Task deletion is available to mentors and admins. The Kanban interface provides an intuitive drag-and-drop experience for status management.

4. Report Submission

The Reports page allows interns to write and submit weekly progress reports in markdown format. Each submission is saved with a timestamp and the intern's identity. Mentors can view all submitted reports, filter them by intern, and access the full markdown content of each report.

5. Mentor Feedback

For each submitted report, mentors can add structured feedback through the feedback module. Feedback entries are linked to the specific report they address, creating a conversation thread between the mentor and intern. Interns can view all feedback they have received on their reports.

6. Data Flow

The frontend communicates with the backend using the centralized api.js service, which attaches JWT tokens to all authorized requests. The backend processes each request through FastAPI route handlers and SQLAlchemy database sessions. Pydantic schemas validate all incoming and outgoing data. Responses are returned as JSON and rendered in the React UI.

Output Screens

The following descriptions represent the main interfaces of the developed InternHub – Remote Internship Progress Dashboard. These screens demonstrate the practical working of authentication, task management, report submission, and the feedback modules.

1. Login Page

The Login page provides a clean authentication interface where users enter their email and password to access the system. On successful authentication, the JWT token is stored and the user is redirected to the Dashboard.

2. Register Page

The Register page allows new users to create an account by entering their name, email, password, and selecting a role (Admin, Mentor, or Intern). Role selection at registration determines the access level and interface features available to the user.

3. Dashboard

The Dashboard provides a personalized overview of internship activities based on the logged-in user's role. Mentors see task and report summaries. Interns see their assigned tasks and report submissions. The DashboardNavbar provides quick navigation links and a logout button.

4. Task Board (Kanban)

The TaskBoard page displays tasks organized into status columns using a Kanban layout. Mentors can create new tasks, assign them to interns, and manage task status. Interns can view their assignments and update task progress. The board provides a visual representation of all active internship tasks.

5. Reports Page

The Reports page allows interns to write and submit weekly progress reports in markdown format. Submitted reports are listed with their timestamps and are accessible for mentor review. Mentors can read the full report content and navigate to the feedback section for each submission.

6. Feedback Section

The Feedback section is accessible per report and allows mentors to add structured responses to intern submissions. Feedback entries are displayed in chronological order, creating a clear communication thread between mentor and intern for each weekly report.

Result and Analysis

The developed system successfully integrates role-based user authentication, Kanban task management, structured report submission, and mentor feedback into one web application. The final result is a usable internship management platform where an organization can register users with defined roles, assign and track tasks visually, collect weekly intern reports, and maintain a structured feedback record.

Observed Outcomes

- The role-based authentication system correctly restricts access to appropriate features for Admin, Mentor, and Intern roles.
- Task creation and assignment through the Kanban board functions correctly, with tasks persisted in the database and visible to the assigned intern.
- The Kanban board accurately represents task status, allowing mentors to monitor progress visually.
- Interns can successfully submit markdown-formatted weekly reports that are stored with timestamps and associated with their account.
- Mentors can view all submitted reports and add structured feedback linked to specific report entries.
- The centralized API service (api.js) correctly attaches JWT tokens to all protected requests, ensuring secure communication between frontend and backend.
- The Dashboard provides relevant role-specific information immediately after login, reducing navigation overhead.
- The SQLite database is automatically created on first startup, confirming that the project requires no manual database configuration.

Analysis

From a functional perspective, the project meets its major objectives. The core modules of authentication, task management, report submission, and feedback are integrated into a

coherent workflow. This indicates that the architecture is stable and that the selected technologies are suitable for the problem domain.

The Kanban task board is one of the most significant functional outcomes of the system. Its visual nature makes task status immediately understandable to both mentors and interns, reducing the need for status update meetings or email check-ins. By providing a shared visual representation of internship work, the system improves transparency and accountability across all participants.

The report submission and feedback cycle adds strong practical value. In many internship programs, progress reporting is informal and inconsistent. By providing a structured markdown-based submission system with a linked feedback mechanism, the project formalizes this workflow and creates a persistent, searchable record of internship activity.

From a software engineering perspective, the project demonstrates the advantages of modular full-stack design. The FastAPI backend is organized into domain-specific routers (auth, tasks, reports, feedback), which makes the codebase readable, maintainable, and easy to extend. The React frontend separates pages, components, and API services cleanly, supporting a consistent user experience across all routes.

Some limitations remain in the current version. The system does not yet support real-time notifications, file attachments, advanced analytics, or email-based alerts. The current scheduling and notification features are manual. These limitations do not reduce the usefulness of the present implementation but identify clear directions for future enhancement.

Overall, the analysis shows that the project successfully fulfills its intended purpose. It is technically stable, functionally meaningful, and practically relevant. The current version provides a strong foundation for future development and demonstrates how full-stack software can effectively solve a real-world coordination problem in remote internship management.

Software Testing

Software testing was carried out to verify the correctness, stability, and usability of all major project modules. The purpose of testing was to ensure that all core functional components work reliably under normal user interaction, including authentication flows, task operations, report submission, and feedback management. Both frontend behavior and backend API responses were evaluated.

1. Testing Approach

- Functional testing: Verifying that each core feature behaves according to expected output.
- API testing: Confirming correct request-response behavior for all backend endpoints.
- UI interaction testing: Checking page navigation, form submission, button actions, and state updates.
- Validation testing: Ensuring incorrect or incomplete inputs are handled properly.
- Role-based access testing: Verifying that route protection and role-specific views work correctly.
- Build verification: Confirming that the frontend project compiles successfully.

2. Test Cases

Test Case	Input / Action	Expected Result	Status
User Registration	Enter name, email, role, password	Account created successfully	Passed
User Login	Enter registered email and password	Token generated, dashboard opened	Passed
Invalid Login	Wrong email or password	Error message shown	Passed
Role-based Routing	Intern accesses mentor-only page	Redirect or access denied	Passed
Create Task	Mentor enters task details and assigns intern	Task saved and visible on Kanban board	Passed
Update Task Status	Intern changes task column on Kanban board	Task status updated in database	Passed
Delete Task	Mentor deletes a task	Task removed from board and database	Passed
Submit Report	Intern writes and submits weekly report	Report saved with timestamp	Passed
View Reports	Mentor opens reports page	All intern reports listed correctly	Passed
Add Feedback	Mentor adds feedback to a report	Feedback saved and visible to intern	Passed
Protected Routes	Access dashboard without login	Redirect to login page	Passed
Frontend Build	Run production build command	Project compiles without errors	Passed

3. Verification Summary

The results of software testing confirm that all major modules of the system function correctly and in coordination with one another. Authentication and role-based access work as expected, task operations on the Kanban board perform correctly, report submission and retrieval are functioning without errors, and the mentor feedback system is operating as designed.

The frontend application was also successfully built for production, confirming that the client-side code is structurally stable. Backend route protection and JWT token validation were verified through repeated functional checks. Error handling for invalid login attempts and unauthorized access was observed to work correctly.

Based on the testing performed, the project can be considered stable, usable, and suitable for academic demonstration. Although more advanced testing methods such as automated unit testing with pytest and Jest, load testing, and security penetration testing can be added in future, the current level of testing is sufficient to confirm that the present implementation meets its functional requirements.

Conclusion

"InternHub – Remote Internship Progress Dashboard" successfully demonstrates how modern web technologies can be used to solve a real-world coordination problem faced by organizations managing remote internship programs.

The project provides a secure, organized, and role-based environment for handling task management, structured report submission, and mentor feedback. Its most valuable contributions are the Kanban-based task tracking interface, which provides visual accountability for all internship work, and the formal report-feedback cycle, which replaces informal communication channels with structured, auditable documentation.

From a technical perspective, the project demonstrates effective use of React, FastAPI, SQLAlchemy, SQLite, JWT authentication, and role-based access control. From a practical perspective, it provides a usable tool for internship coordinators and a scalable base for future educational or professional workflow software.

The project also highlights the importance of combining structured data management with role-appropriate interfaces. Rather than providing a generic tool, the application actively separates the concerns of Admins, Mentors, and Interns, ensuring that each participant sees exactly what

is relevant to their role. This improves usability, reduces confusion, and makes the system feel purposeful rather than generic.

Another important outcome is the learning value it provides from a software development perspective. It demonstrates the successful integration of frontend design, backend API development, database connectivity, authentication, and access control into one complete system. Therefore, the project is not only useful as an end-user application, but also as a strong example of applied full-stack development in a professional workflow management context.

The present version of the system already fulfills its core purpose effectively, while also leaving clear opportunities for further enhancement. Features such as real-time notifications using WebSockets, email alerts for task assignments and report submissions, advanced analytics dashboards, file attachment support, calendar synchronization, and AI-based intern performance insights can be added in future versions to make the platform even more intelligent and comprehensive.

In conclusion, the project meets its objectives and serves as a meaningful example of full-stack software development applied to remote work and internship management. It provides both practical value to organizations and technical value as a project study, while establishing a solid foundation for future growth.

References

- FastAPI Documentation. https://fastapi.tiangolo.com/
- React Documentation. https://react.dev/
- react-kanban Documentation. https://github.com/asseinfo/react-kanban
- SQLAlchemy Documentation. https://www.sqlalchemy.org/
- SQLite Documentation. https://www.sqlite.org/docs.html
- Pydantic Documentation. https://docs.pydantic.dev/
- python-jose Documentation. https://python-jose.readthedocs.io/
- JSON Web Token Introduction. https://jwt.io/introduction
- Uvicorn Documentation. https://www.uvicorn.org/
- Node.js Documentation. https://nodejs.org/en/docs/